# CS1004: COMPUTER ORGANIZATION AND ARCHITECTURE

CSE, SNU,Chennai

# Textbooks / References

- ***TEXTBOOK:***
- Carl Hamacher, Zvonko Vranesic, Safwat Zaky, “Computer Organization”, McGraw Hill Education, Fifth Edition, 2011

# Number Representation

SHIV NADAR UNIVERSITY CHENNAI

## Numbers & Arithmetic operations

- Sign & magnitude
- 1’s complement
- 2’s complement



SHIV NADAR UNIVERSITY CHENNAI

| $b_3b_2b_1b_0$ | Sign & Magnitude | 1's Complement | 2's Complement |
|---|---|---|---|
| 0111 | +7 | +7 | +7 |
| 0110 | +6 | +6 | +6 |
| 0101 | +5 | +5 | +5 |
| 0100 | +4 | +4 | +4 |
| 0011 | +3 | +3 | +3 |
| 0010 | +2 | +2 | +2 |
| 0001 | +1 | +1 | +1 |
| 0000 | +0 | +0 | +0 |
| 1000 | -0 | -7 | -8 |
| 1001 | -1 | -6 | -7 |
| 1010 | -2 | -5 | -6 |
| 1011 | -3 | -4 | -5 |
| 1100 | -4 | -3 | -4 |
| 1101 | -5 | -2 | -3 |
| 1110 | -6 | -1 | -2 |
| 1111 | -7 | -0 | -1 |

# Exercise problems:

a. 0010 (+2)
+0011 (+3)
0101 (+5)

b. 0100 (+4)
+ 1010 (-6)
_____

c. 1011 (-5)
+1110 (-2)
____

d. 0111 (+7)
+1101 (-3)

e. 1101 (-3)
-1001 (-7)
0100 (+4)

f. 0010 (+2)
-0100 (+4)

g. 0110 (+6)
-0011 (+3)

h. 1001 (-7)
-1011 (-5)

i. 1001 (-7)
-0001 (+1)

# Problems Solution:

SHIV NADAR UNIVERSITY CHENNAI

a. 0010 (+2)
+0011 (+3)
0101 (+5)

B. 0100 (+4)
+ 1010 (-6)
1110 (-2)

C. 1011 (-5)
+1110 (-2)
1001 (-7)

D. 0111 (+7)
+1101 (-3)
0100 (+4)

e. 1101 (-3)
-1001 (-7)
0100 (+4)

f. 0010 (+2)
-0100 (+4)
1110 (-2)

g. 0110 (+6)
-0011 (+3)
0011 (+3)

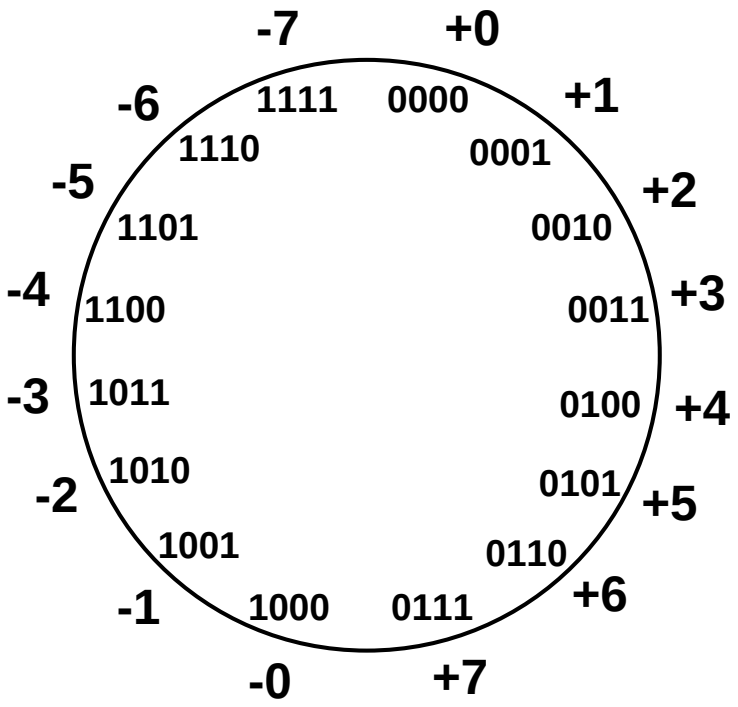
h. 1001 (-7)
-1011 (-5)
1110 (-2)

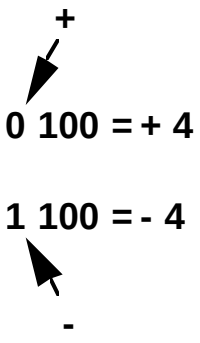
i. 1001 (-7)
-0001 (+1)
1000 (-8)



SHIV NADAR UNIVERSITY CHENNAI

# Sign and Magnitude Representation



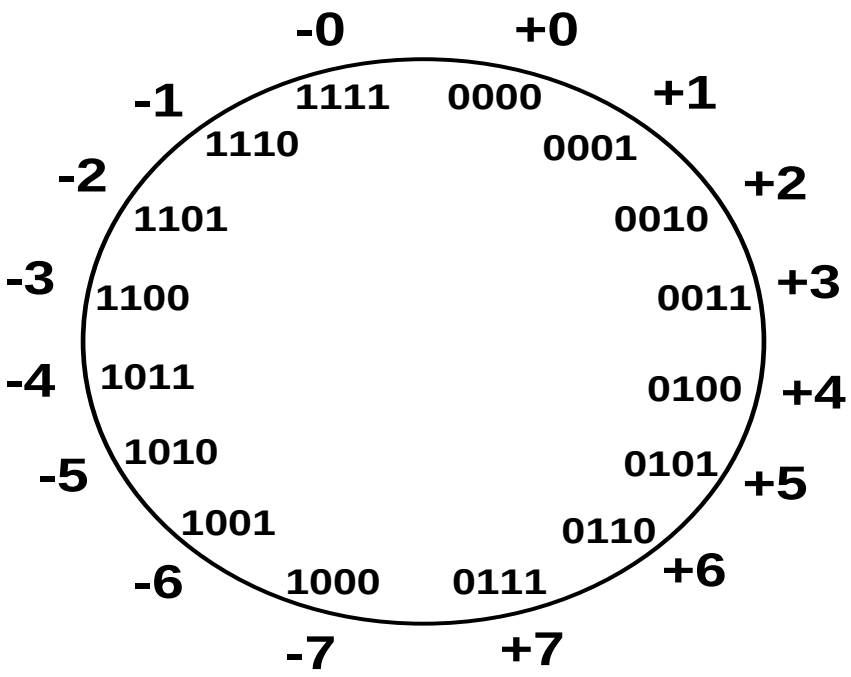

High order bit is sign: 0 = positive (or zero), 1 = negative

Three low order bits is the magnitude: 0 (000) thru 7 (111)

Number range for n bits = $+/-2^{n-1}$ -1

Two representations for 0

# One's Complement Representation




+

0 100 = + 4

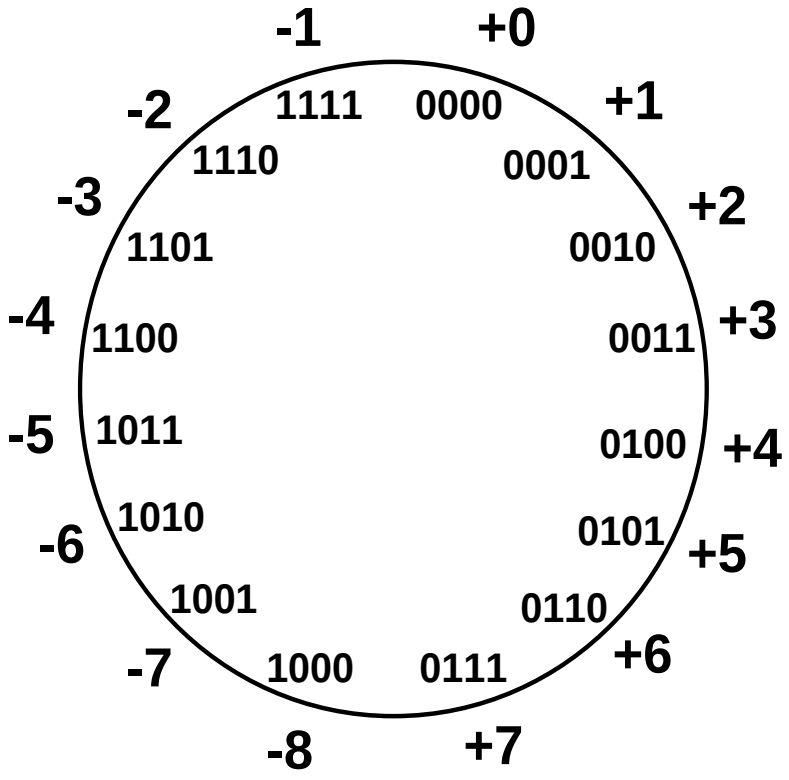
1 011 = - 4

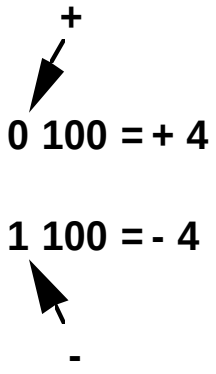
-

- Subtraction implemented by addition & 1's complement
- Still two representations of 0!  This causes some problems
- Some complexities in addition

# Two's Complement Representation

*like 1's comp except shifted one position clockwise*




- Only one representation for 0
- One more negative number than positive number

# Rules

- To add 2 nos, add their n-bit representations ignoring carry out signal from the most significant bit(MSB) position.
- The sum will be algebraically correct value in 2's complement representation as long as the answer is range $-2^{n-1}$ through $+2^{n-1}-1$.
- To subtract 2 nos x and y, to perform x-y, form the 2's complement of y and then add it to x.
- The result will be algebraically correct value in 2's complement representation system if answer is in range $-2^{n-1}$ through $+2^{n-1}-1$.
- For +ve and –ve nos , discard the carry out.

# 2's-Complement Add and Subtract Operations

| | | | | | | |
|---|---|---|---|---|---|---|
| (a) | 0 0 1 0 | (+2) | | (b) | 0 1 0 0 | (+4) |
| | + 0 0 1 1 | (+3) | | | + 1 0 1 0 | (- 6) |
| | 0 1 0 1 | (+5) | | | 1 1 1 0 | (- 2) |
| (c) | 1 0 1 1 | (- 5) | | (d) | 0 1 1 1 | (+7) |
| | + 1 1 1 0 | (- 2) | | | + 1 1 0 1 | (- 3) |
| | 1 0 0 1 | (- 7) | | | 0 1 0 0 | (+4) |
| (e) | 1 1 0 1 | (- 3) | ⇒ | | 1 1 0 1 | |
| | - 1 0 0 1 | (- 7) | | | + 0 1 1 1 | |
| | | | | | 0 1 0 0 | (+4) |
| (f) | 0 0 1 0 | (+2) | ⇒ | | 0 0 1 0 | |
| | - 0 1 0 0 | (+4) | | | + 1 1 0 0 | |
| | | | | | 1 1 1 0 | (- 2) |
| (g) | 0 1 1 0 | (+6) | ⇒ | | 0 1 1 0 | |
| | - 0 0 1 1 | (+3) | | | + 1 1 0 1 | |
| | | | | | 0 0 1 1 | (+3) |
| (h) | 1 0 0 1 | (- 7) | ⇒ | | 1 0 0 1 | |
| | - 1 0 1 1 | (- 5) | | | + 0 1 0 1 | |
| | | | | | 1 1 1 0 | (- 2) |
| (i) | 1 0 0 1 | (- 7) | ⇒ | | 1 0 0 1 | |
| | - 0 0 0 1 | (+1) | | | + 1 1 1 1 | |
| | | | | | 1 0 0 0 | (- 8) |
| (j) | 0 0 1 0 | (+2) | ⇒ | | 0 0 1 0 | |
| | - 1 1 0 1 | (- 3) | | | + 0 0 1 1 | |
| | | | | | 0 1 0 1 | (+ 5) |

Figure 2.4. 2's-complement Add and Subtract operations.

## Binary Fraction to Decimal Fraction

- Example: What is the binary value **0.011010** in decimal ?

| . | 0 | 1 | 1 | 0 | 1 |
|---|---|---|---|---|---|
| 32 | 16 | 8 | 4 | 2 | 1 |

Sum = **8+4+1 = 13**

Answer: 13 / 32 = 0.40625

- Example: What is **0 . 0 0011 0011 00** in decimal ?

Answer: **(32+16+2+1) / 512 = 51 / 512 = 0.099609375**



## Decimal Fraction to Binary Fraction

- Example: What is $0.6875_{10}$ in binary ?

0.6875 * 2 = 1 .3750
0.3750 * 2 = 0 .7500
0.7500 * 2 = 1 .5000
0.5000 * 2 = 1 .0000
0.0000 * 2 = 0

Answer: $0.1011_2$

- Example What is $0.1_{10}$ in binary ?

# Floating Pointing Numbers

**Scientific Notation**

| | | | |
|---|---|---|---|
| Number | = | $M \times 10^E$ | ← Decimal |
| Number | = | $M \times 2^E$ | ← Binary |

- M is the Mantissa (or Significand or Fraction or Argument )
- E is the Exponent (or Characteristic)
- 10 (or for binary, 2) is the Radix (or Base)

- Digits (bits) in Exponent -> Range (Bigness/Smallness)
- Digits (bits) in Mantissa -> Precision (Exactness)

# Floating Point Numbers



- Sign – Represent sign of  number (0-positive, 1-negative)
- Exponent ( Bias is a constant added to exp to ensure  stored value is positive)
- Mantissa – fractional part

What do we need to store when representing floating point numbers in a computer?

$$x = \pm 1.f \times 2^{m}$$

$$x = \underbrace{\boxed{\pm}}_{\text{sign}}\underbrace{\boxed{m}}_{\text{exponent}}\underbrace{\boxed{f}}_{\text{significand}}$$

**IEEE-754 Single precision (32 bits):**

$x =$

| | | |
|---|---|---|
| | | |

**IEEE-754 Double precision (64 bits):**

$x =$

| | | |
|---|---|---|
| | | |

# IEEE-754 Single Precision (32-bit)

$$x = (-1)^{s}\, 1.f \times 2^{m}$$

# Single Precision Format (32-bit)

| Sign S | Exponent E | Significand F |
|---|---|---|
| 1 bit | 8 bits | 23 bits |

The mantissa is called the SIGNIFICAND in the IEEE standard

Value represented = $\pm 1.F \times 2^{E-127}$ $\quad 127 = 2^{8-1} - 1$

# Double Precision Format (64-bit)

| Sign S | Exponent E | Significand F |
|---|---|---|
| 1 bit | 11 bits | 52 bits |

Value represented = $\pm 1.F \times 2^{E-1023}$

$1023 = 2^{11-1} - 1$

# Normalization

| Number | Normalised Binary |
|---|---|
| $100.01 \times 2^{1}$ | $1.0001 \times 2^{3}$ |
| $1010.11 \times 2^{2}$ | $1.01011 \times 2^{5}$ |
| $0.00101 \times 2^{-2}$ | $1.01 \times 2^{-5}$ |
| $1100101 \times 2^{-2}$ | $1.100101 \times 2^{+4}$ |

What is 42.6875 in IEEE Single Precision Format?

First convert to a binary number: **42.6875 = 10_1010 . 1011**

Next normalise: **1 . 0101_0101_1 x $2^5$**

Significand field is therefore: **0101_0101_1000_0000_0000_000**

Exponent field is (5+127=132): 1000_0100

Value in IEEE Single Precision is:

| Sign | Exponent | Significand |
|---|---|---|
| **0** | 1000_0100 | **0101_0101_1000_0000_0000_000** |

**0100__0010__0 010__1010__1100__0000__0000__0000**

# Find Decimal Value from the 32 bit pattern

- 0 10000100 01010101100000000000000
- E=132 (10000100 )
- E’=132-127=5
- S=0 (+ve)
- +/-(1.M)*2^E’
- +(1. 0010101011)*2^5
- (1+1/2^2+1/2^4+1/2^6+1/2^8+1/2^9)*2^5
- (32+8+2+11/16)
- 42.6875

# IEEE 754-Floating Point Number

- Half Precision (16 bit): 1 sign bit, 5 bit exponent, and 10 bit mantissa
- Single Precision (32 bit): 1 sign bit, 8 bit exponent, and 23 bit mantissa
- Double Precision (64 bit): 1 sign bit, 11 bit exponent, and 52 bit mantissa
- Quadruple Precision (128 bit): 1 sign bit, 15 bit exponent, and 112 bit mantissa

# Floating Point Arithmetic

- Step1:Shift the decimal point of the smaller number to the left until the exponents are equal.
- Step2:Add/sub the numbers with decimal points aligned
- Step3:Normalize the result.

# Floating point Arithmetic

- 2.25* 10^0
- 1.340625*10^2
- Align exponent

  0.0225*10^2

$$
\begin{array}{rll}
 & 0.0225 & \times 10^2 \\
+ & 1.340625 & \times 10^2 \\
\hline
 & 1.363125 & \times 10^2
\end{array}
$$

**Normalize the result**

**0.1363125* 10 ^3**

$0.13 \times 10^{2}$

$0.23 \times 10^{3}$

$0.013 \times 10^{3}$

$0.23 \times 10^{3}$

Addition $\boxed{0.243 \times 10^{3}}$

## Addition

$0.5372400 \times 10^{2}$ ✓

$0.1580000 \times 10^{-1}$

Step-1

$0.1580000 \times 10^{-1} \times 10^{3}$

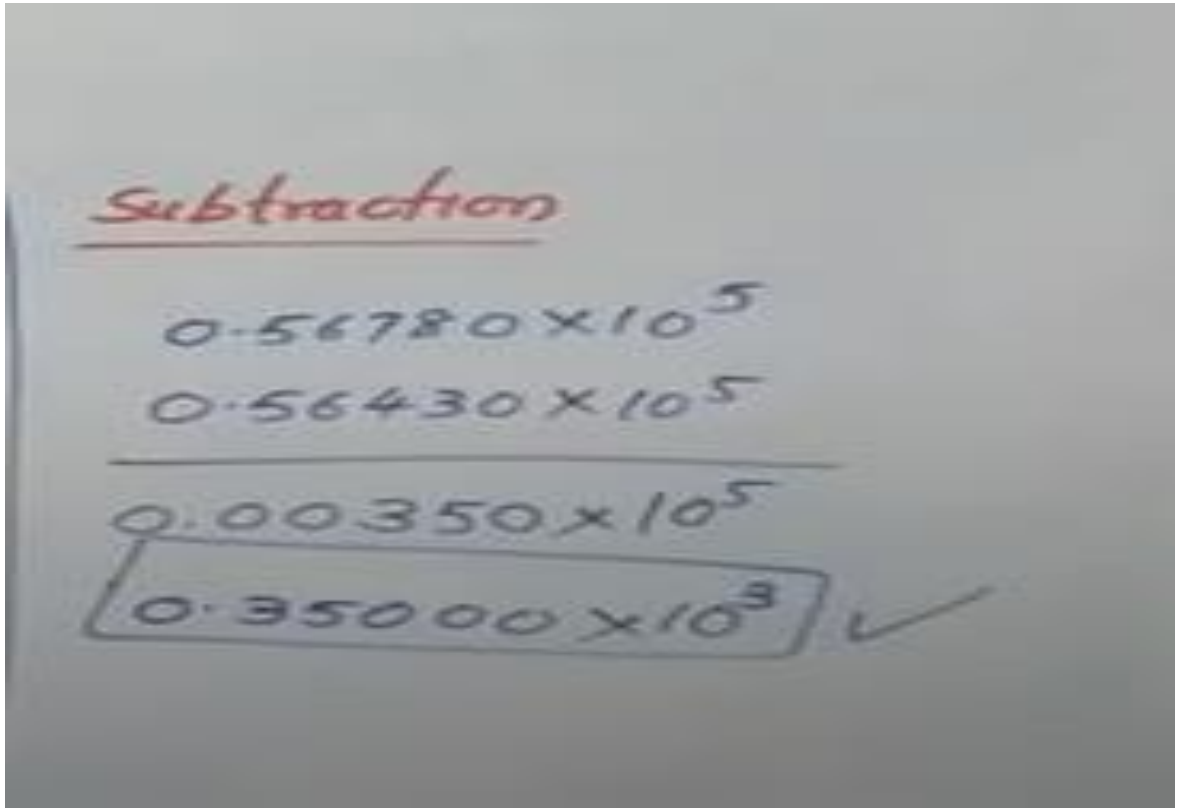
$= 0.0001580 \times 10^{2}$ ✓

$0.5372400 \times 10^{2}$

$0.0001580 \times 10^{2}$

$0.5373980 \times 10^{2}$

## Subtraction


Subtraction
0.56780 × 10^5
0.56430 × 10^5
0.00350 × 10^5
0.35000 × 10^3

# Multiplying Floating Point Numbers

## Multiplication in Scientific Notation

$$3.5 \times 10^2$$
$$5.25 \times 10^4$$

$$(3.5 \times 5.25)\,(10^2 \times 10^4)$$

$$\boxed{18.375 \times 10^6}$$

# Multiplying Floating Point Numbers

## Multiplication in Scientific Notation

$$3.5 \times 10^2$$
$$5.25 \times 10^4$$

$$(3.5 \times 5.25)\,(10^2 \times 10^4)$$

$$\boxed{1.8375 \times 10^7}$$

# Multiplying Floating Point Numbers

## Multiplication in Scientific Notation

$$1.11 \times 2^3$$
$$1.0101 \times 2^6$$

$$(1.11 \times 1.0101)\ (2^3 \times 2^6)$$

$$1.0010011 \times 2^{10}$$

# Multiplication of floating-point numbers in IEEE 754 format

$$X = -0.3_{10} = 0.01\overline{0011}_2 = 1.\overline{0011}_2 \times 2^{-2_{10}}$$

| 1 | 01111101 | 00110011001100110011010 |
|---|---|---|

$$Y = 500.25_{10} = 111110100.01_2 = 1.1111010001_2 \times 2^{8_{10}}$$

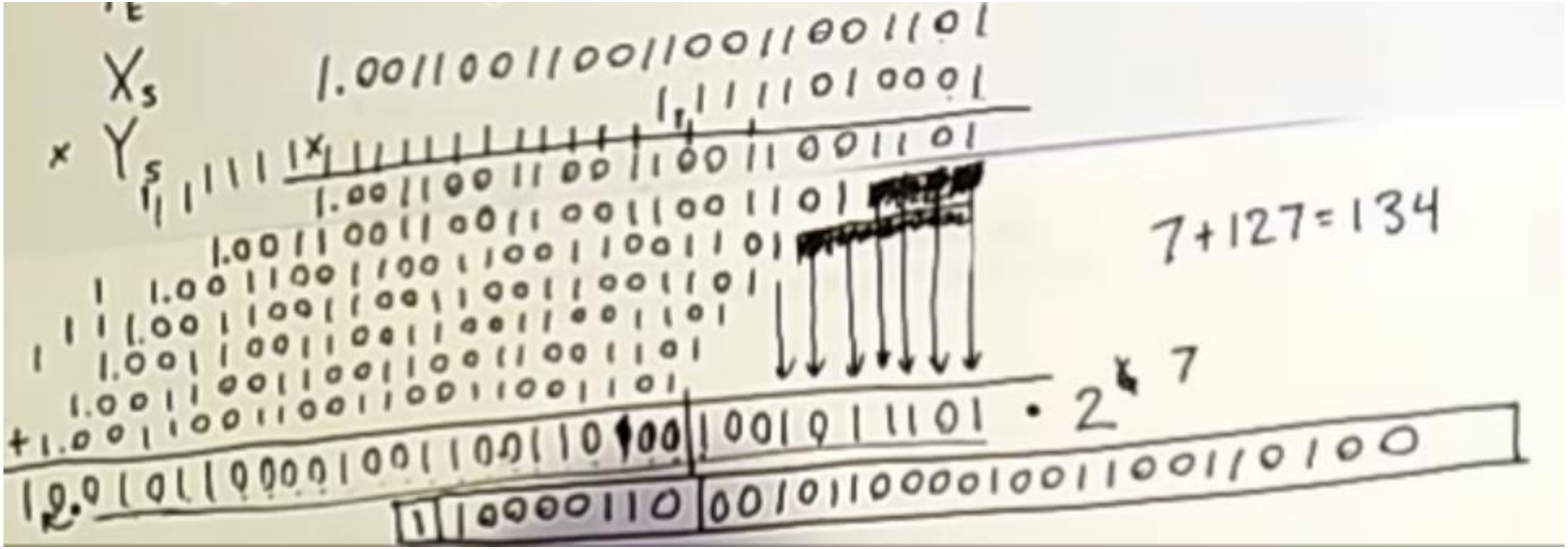
| 0 | 10000111 | 11110100010000000000000 |
|---|---|---|

$$X \cdot Y = (X_s \cdot Y_s) \cdot 2^{X_E + Y_E} = (X_s \cdot Y_s) \cdot 2^6$$

$X_E = -2$ $\quad$ $X_s = 1.00110011001100110011010$

$Y_E = 8$ $\quad$ $Y_s = 1.1111010001$

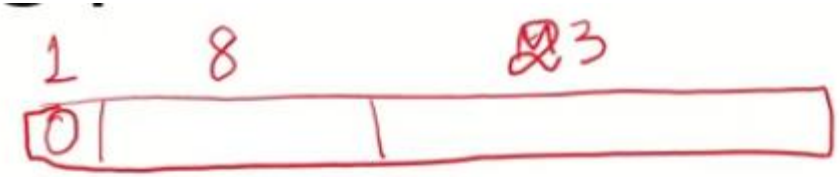
# Multiplication of floating-point numbers in IEEE 754 format



# Addition of floating-point numbers in IEEE 754 format

Add Rule: A + B

- Choose the number with the smaller exponent
  - Shift its mantissa right a number of steps equal to the difference in exponents.
- Set the exponent of the result equal to the larger exponent.
- Perform addition on the mantissas and determine the sign of the result.
- Normalize the resultant mantissa if needed. (1.m format)
  - Initial exponent of result needs to be adjusted according to the normalization of mantissa.




- 9.75 + 18.5625

9.75: 01001.110 = 1.001110 x $2^3$ → 130

$0.1001110 \times 2^4$ → 131

18.5625: 010010.1001 = 1.00101001 x $2^4$ → 131

$2^4$ → 131

$$
\begin{array}{r}
0.10011100 \\
1.00101001 \\
\hline
1.11000101 \rightarrow
\end{array}
$$

$131 + 4 \rightarrow 131$

Result: $1.11000101 \times 2^{131}$

| | | |
|---|---|---|
| 0 | 131 | 11000101000... |

# Subtraction of floating-point numbers in IEEE 754 format

- Subtract rule:
  - Choose the number with the smaller exponent
    - Shift its mantissa right a number of steps equal to the difference in exponents.
  - Set the exponent of the result equal to the larger exponent.
  - Perform subtraction on the mantissas and determine the sign of the result.
  - Normalize the resultant mantissa if needed. (1.m format)
    - Initial exponent of result needs to be adjusted according to the normalization of mantissa.

# Subtraction of floating-point numbers in IEEE 754 format

- 9.75 - 0.5625

9.75: 01001.110 = $1.001110 \times 2^{3}$

0.5625: 0.1001 = $1.001 \times 2^{-1}$

= $0.0001001 \times 2^{3}$

excess-127

3 → 130

# Subtraction of floating-point numbers in IEEE 754 format



$$
\begin{array}{r}
1.0011100 \\
0.0001001 \\
\hline
1.0010011
\end{array}
$$

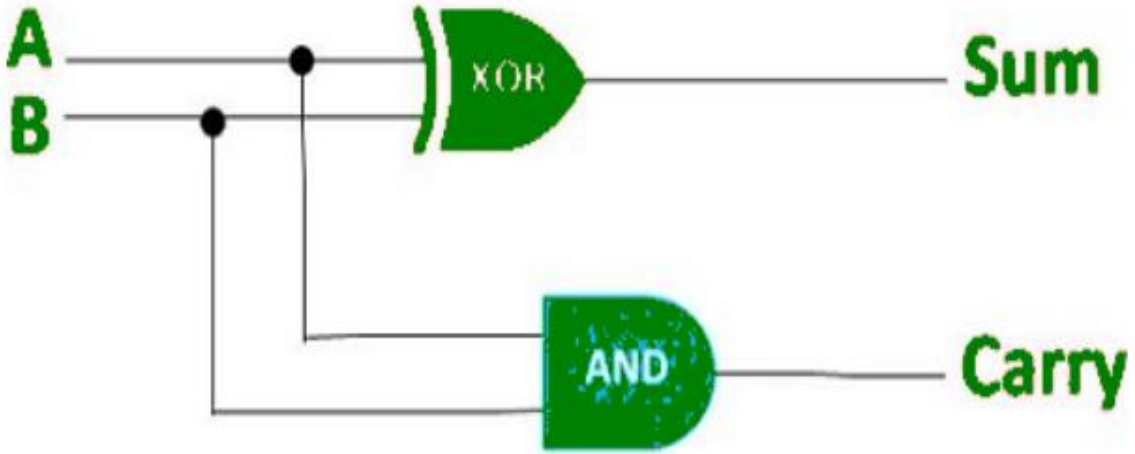
Result: $1.\boxed{0010011} \times 2^{3}$ → 130

1.m

| S | E | M |
|---|---|---|
| 0 | ← → | 00100110 ... 0 ← 23 → |

# Half Adder

| Truth Table | | | |
|---|---|---|---|
| Input | | Output | |
| A | B | Sum | Carry |
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |



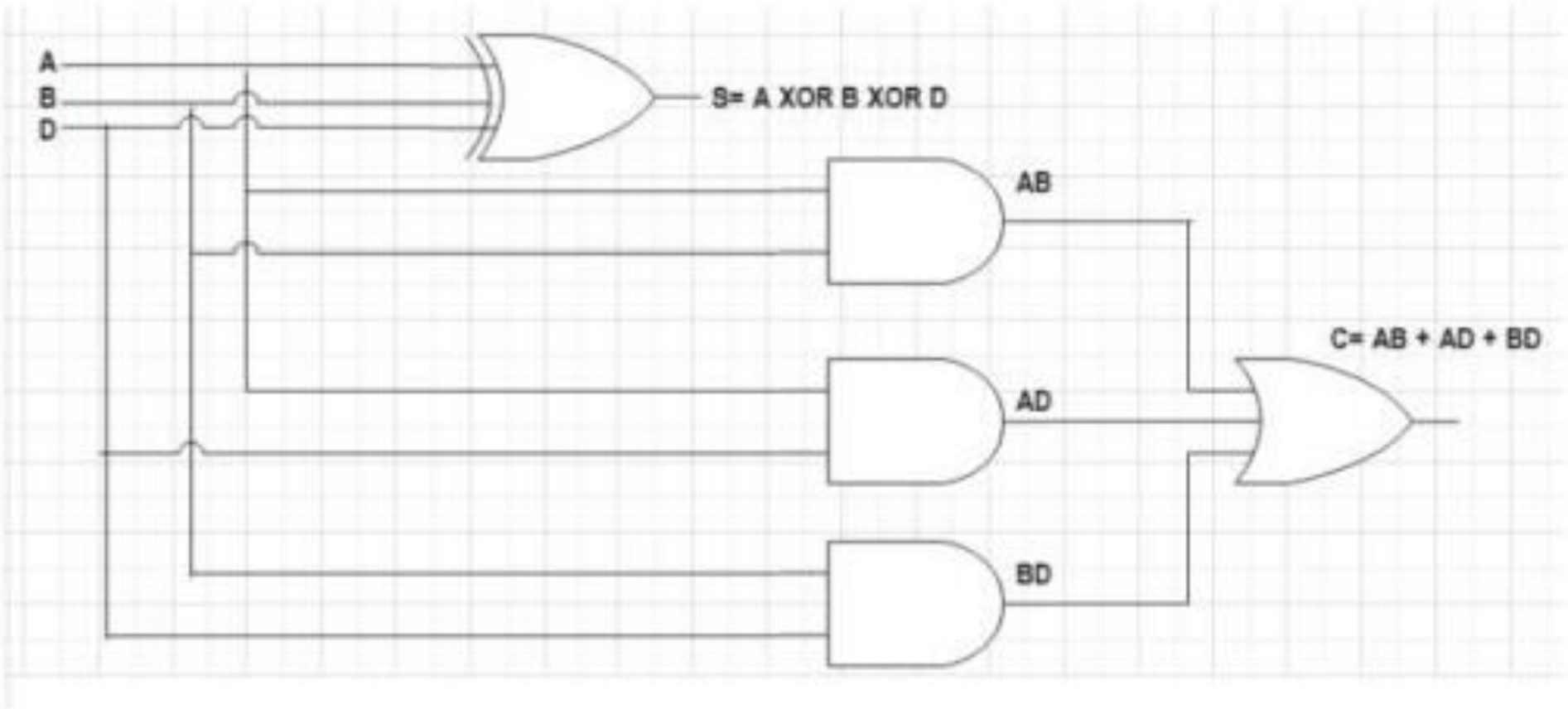
# Half Adder

The output equations of the half adder are –

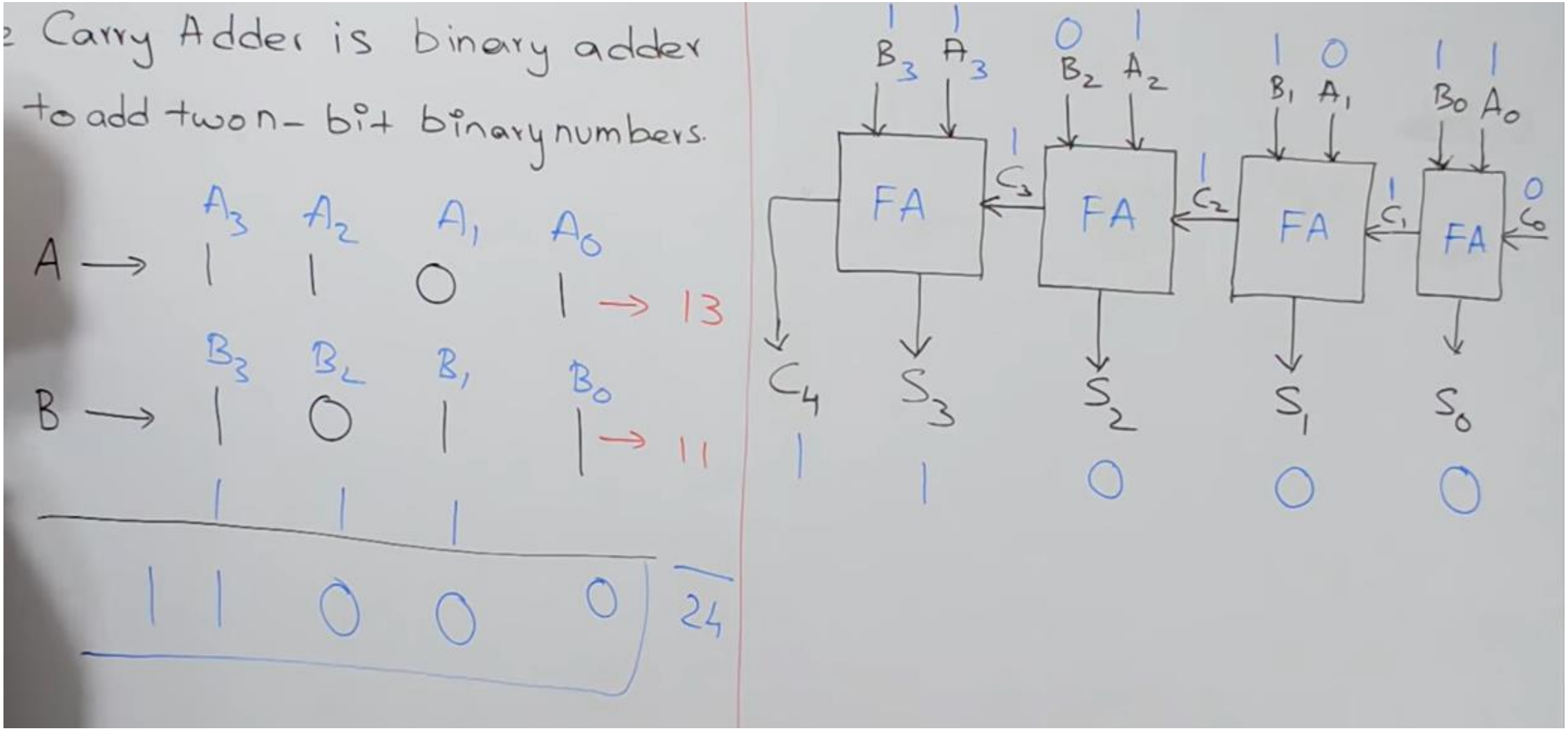
$$Sum, S = A \oplus B$$

$$Carry, C = A.B$$

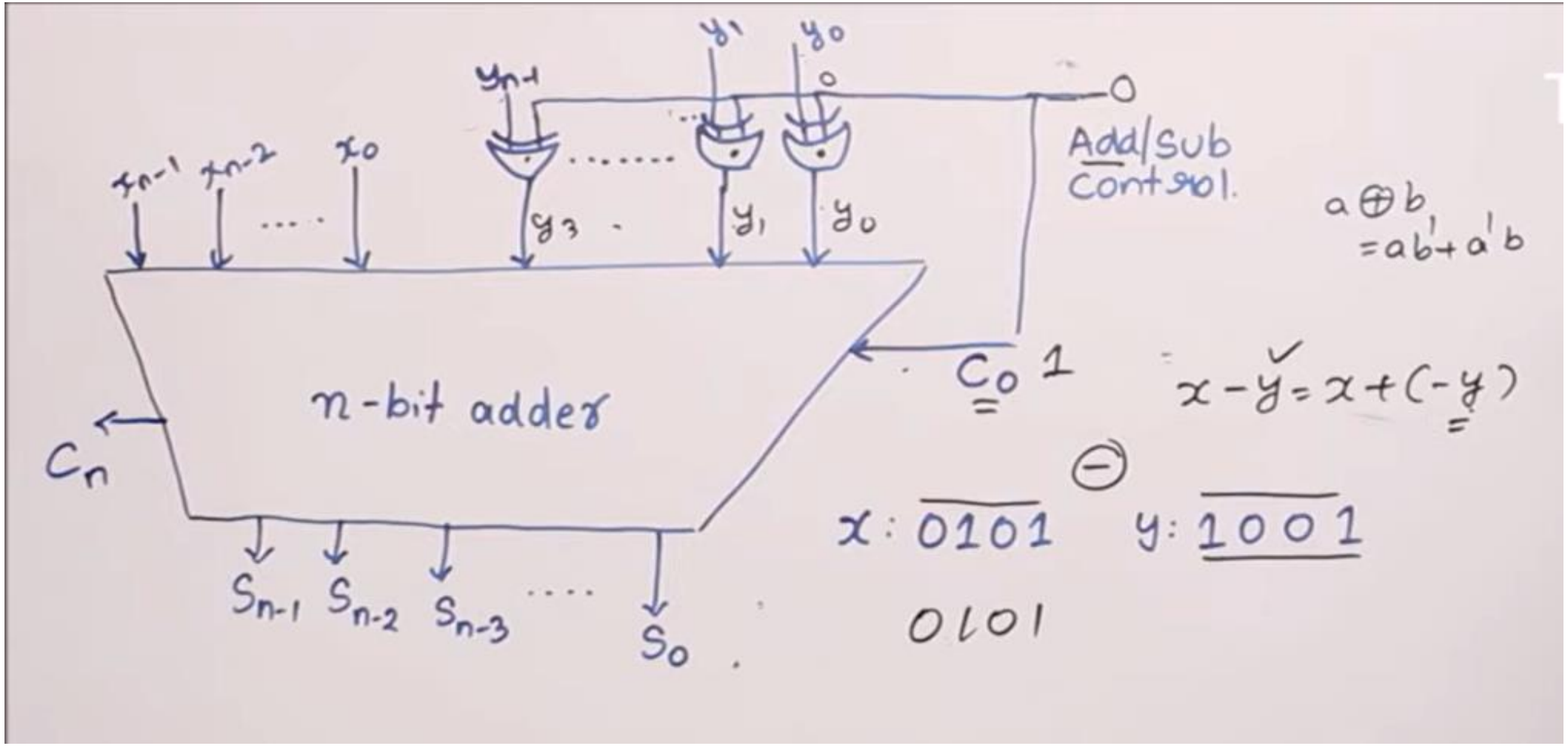
# Full Adder

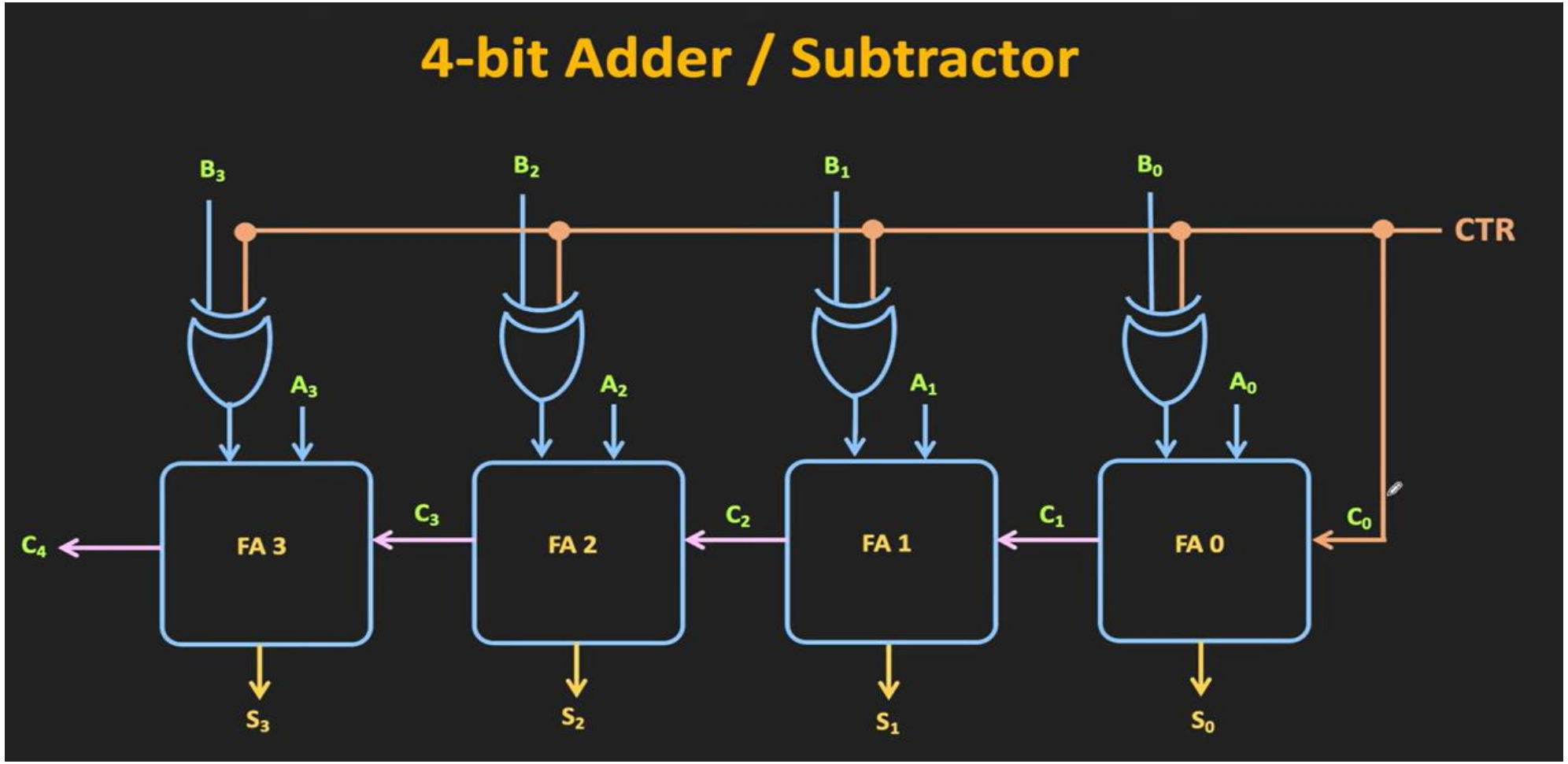


# Ripple Carry Adder



# Adder/Subtractor Circuit




4-bit Adder / Subtractor
B3
B2
B1
B0
CTR
A3
A2
A1
A0
C4
FA 3
C3
FA 2
C2
FA 1
C1
FA 0
C0
S3
S2
S1
S0

# Carry Look Ahead Adder

SHIV NADAR UNIVERSITY CHENNAI

**Truth Table:**

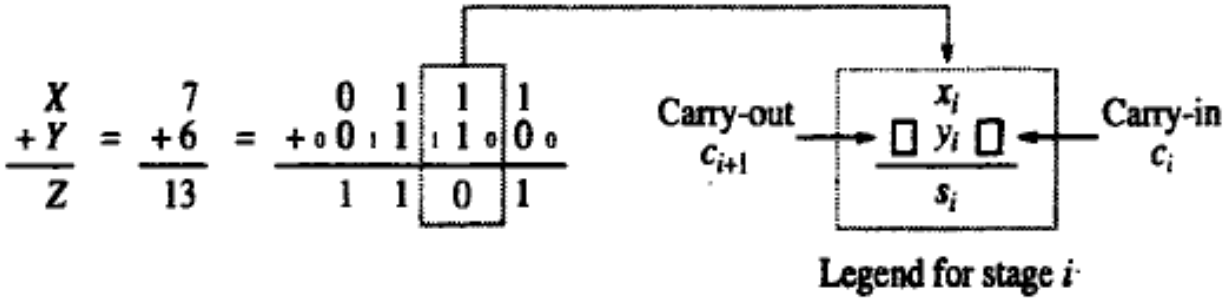
| Input | | | Output | |
|---|---|---|---|---|
| A | B | Cin | Sum | Carry |
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

| $x_i$ | $y_i$ | Carry-in $c_i$ | Sum $s_i$ | Carry-out $c_{i+1}$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

$$s_i = \bar{x}_i\bar{y}_i c_i + \bar{x}_i y_i \bar{c}_i + x_i \bar{y}_i \bar{c}_i + x_i y_i c_i = x_i \oplus y_i \oplus c_i$$

$$c_{i+1} = y_i c_i + x_i c_i + x_i y_i$$

Example:



# Carry Look Ahead Adder



A fast adder circuit must speed up the generation of the carry signals. The logic expressions for $s_i$ (sum) and $c_{i+1}$ (carry-out) of stage $i$ (see Figure 6.1) are

$$s_i = x_i \oplus y_i \oplus c_i$$

and

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

Factoring the second equation into

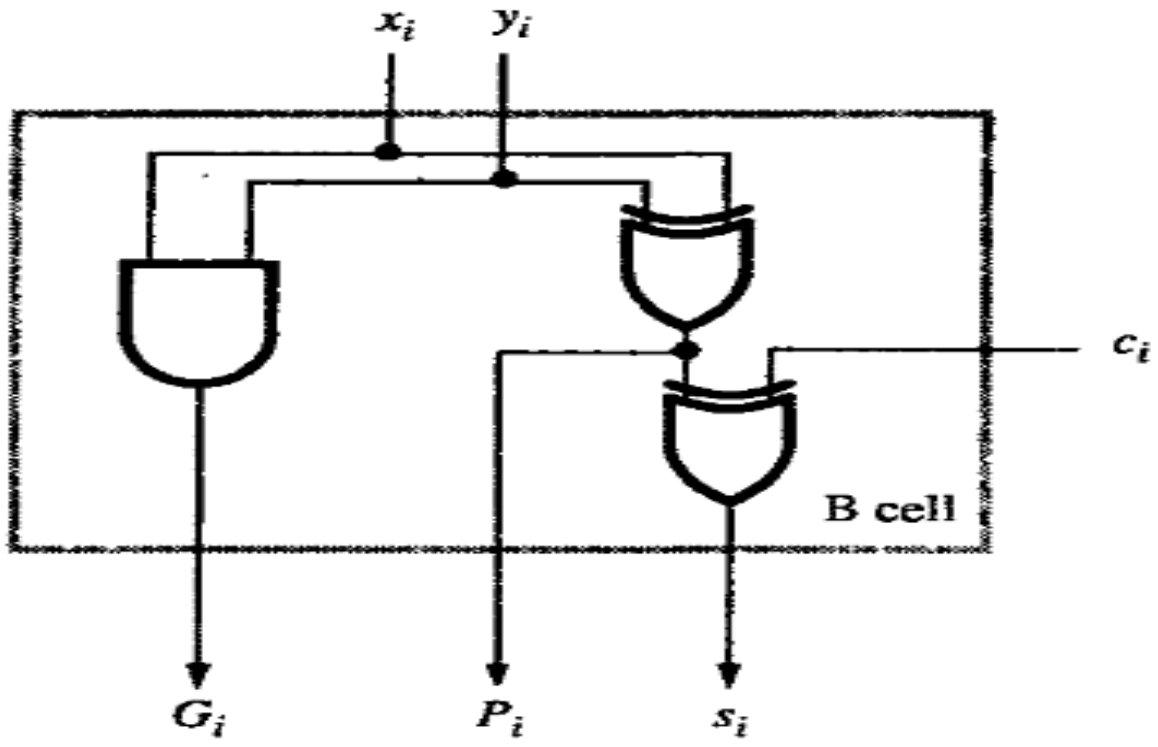
$$c_{i+1} = x_i y_i + (x_i + y_i) c_i$$

we can write

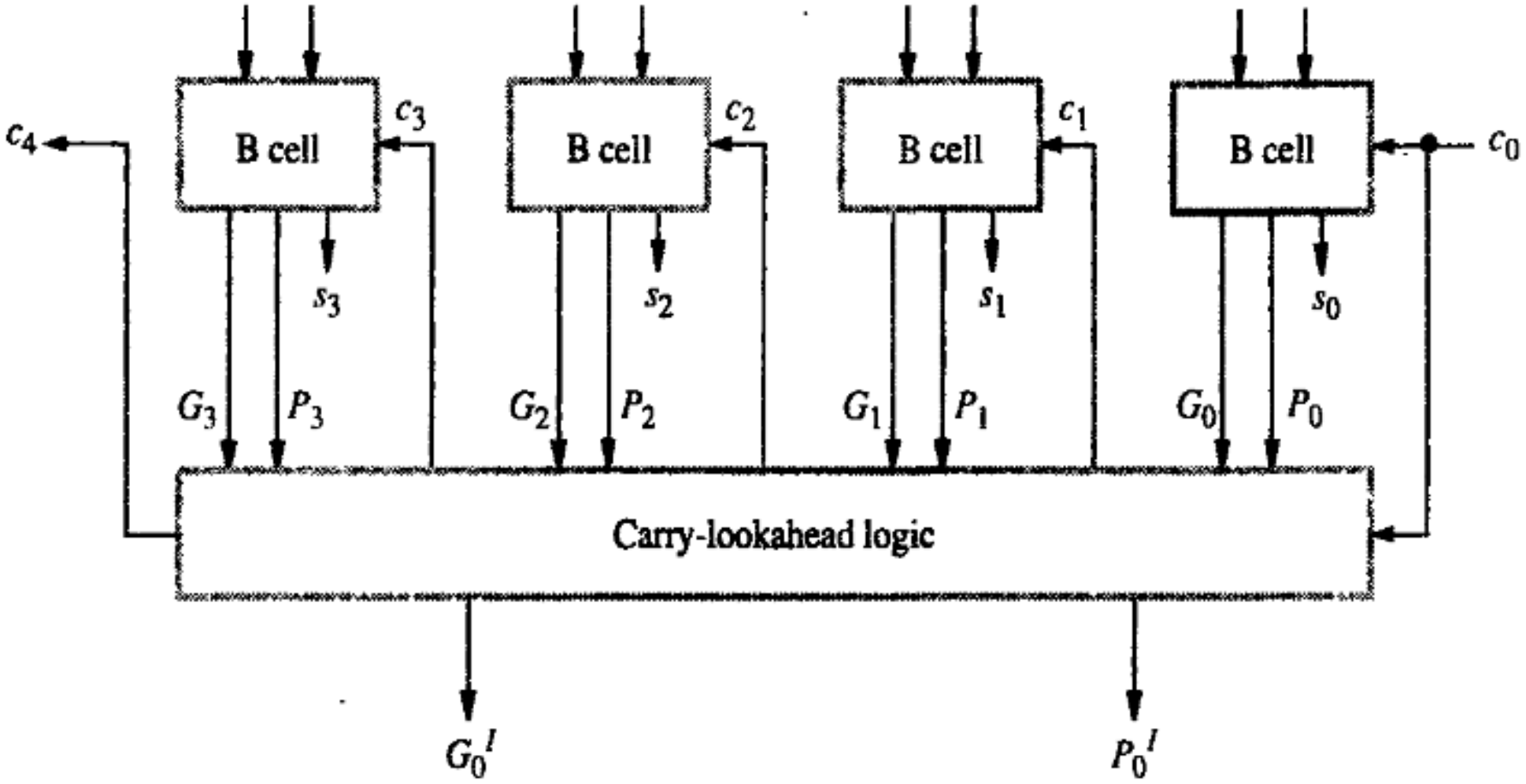
$$c_{i+1} = G_i + P_i c_i$$

where

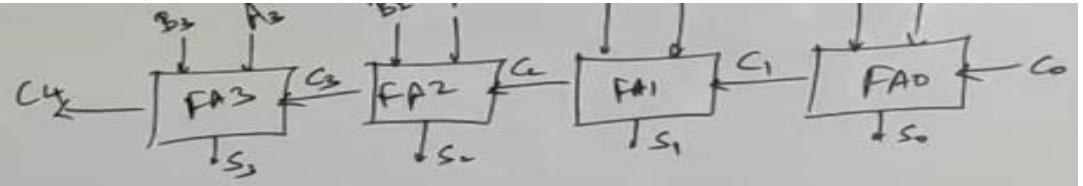
$$G_i = x_i y_i \quad \text{and} \quad P_i = x_i + y_i$$




(a) Bit-stage cell


B cell
B cell
B cell
B cell
$c_4$
$c_3$
$c_2$
$c_1$
$c_0$
$s_3$
$s_2$
$s_1$
$s_0$
$G_3$
$P_3$
$G_2$
$P_2$
$G_1$
$P_1$
$G_0$
$P_0$
Carry-lookahead logic
$G_0^I$
$P_0^I$

# ARRY LOOKAHEAD ADDER




$C_0$ = input Carry

$C_1 = G_0 + P_0 C_0$

$C_2 = G_1 + P_1 C_1 = G_1 + P_1 (G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$

$C_3 = G_2 + P_2 C_2 = G_2 + P_2 (G_1 + P_1 G_0 + P_1 P_0 C_0) = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$

$P_i \Rightarrow$ Carry Propogate

$P_i = A_i \oplus B_i$

$G_i = A_i B_i$

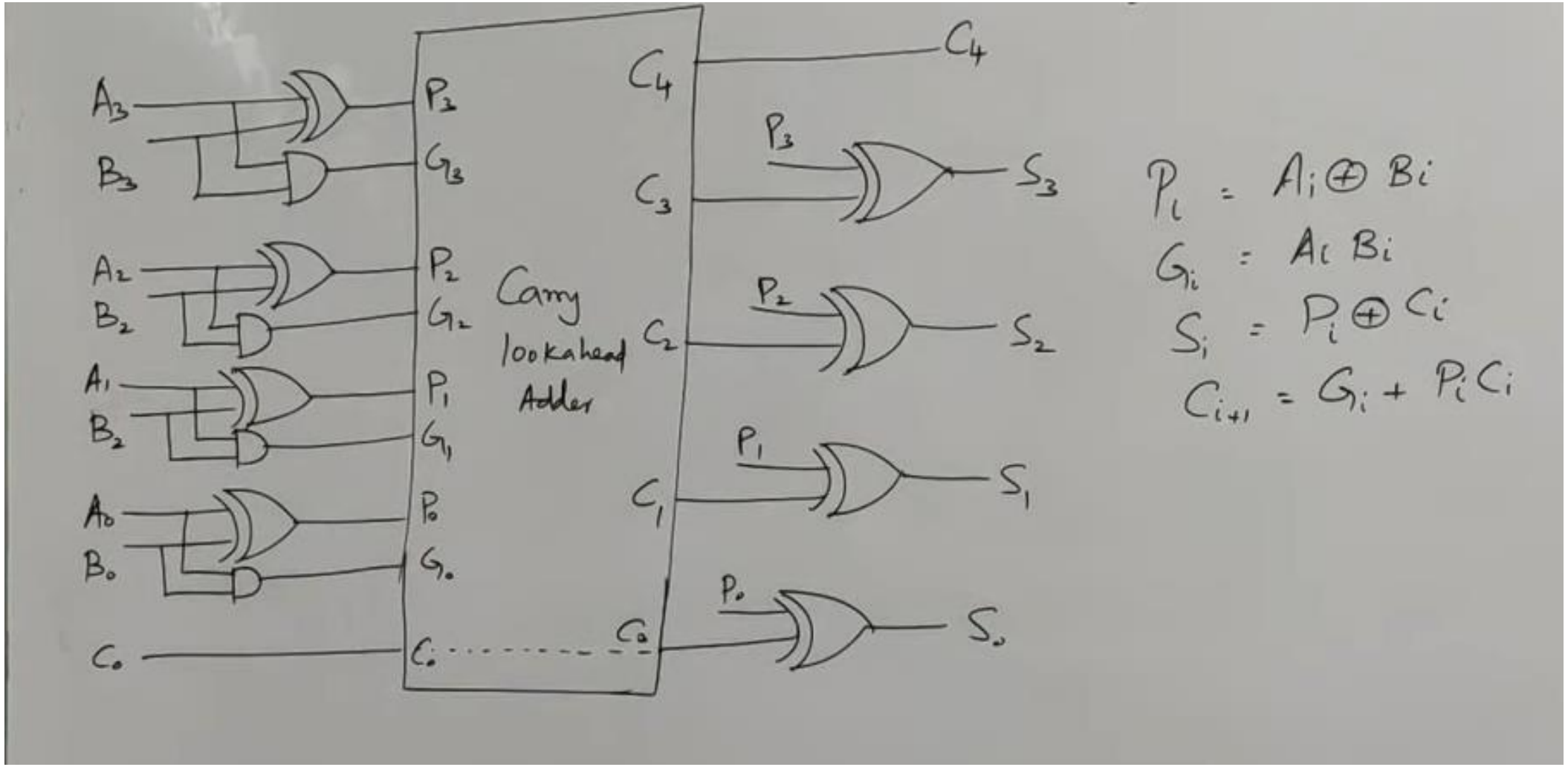
$G_0 = A_0 B_0$

$P_0 = A_0 \oplus B_0$

$G_i \Rightarrow$ Carry Generator

$S_i = P_i \oplus C_i$

$C_{i+1} = G_i + P_i C_i$




$P_i = A_i \oplus B_i$

$G_i = A_i B_i$

$S_i = P_i \oplus C_i$

$C_{i+1} = G_i + P_i C_i$

# THANK YOU !!!